

Day04-系统服务SSH配置网络管理

课程目标

- 理解systemd和服务管理
 - 掌握systemctl命令的使用
 - 学会配置SSH服务
 - 掌握SSH密钥认证
 - 理解网络基本概念
 - 掌握nmcli网络配置
 - 学会配置主机名和DNS
-

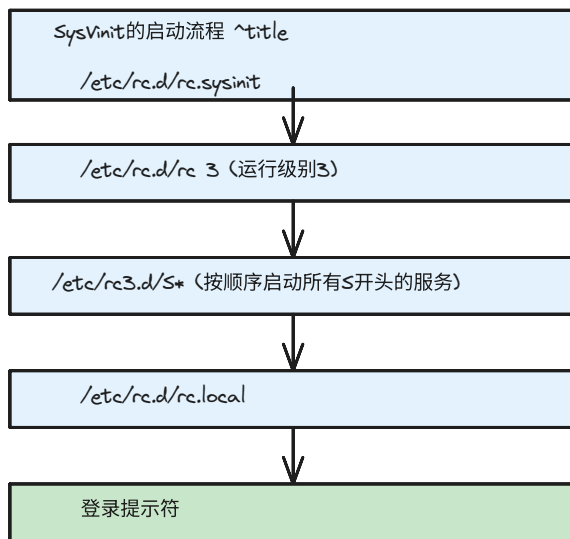
上午 - 系统服务与SSH配置

课前故事：Linux初始化系统的进化

1970-1990年代：SysVinit时代

在Linux早期，系统启动使用的是**SysVinit**（System V init）。这是一个基于串行执行的初始化系统：

SysVinit的启动流程



问题：

1. 串行启动，速度慢
2. 脚本复杂，难以维护
3. 无法处理现代硬件（热插拔）
4. 无法跟踪服务的资源使用

流程说明：

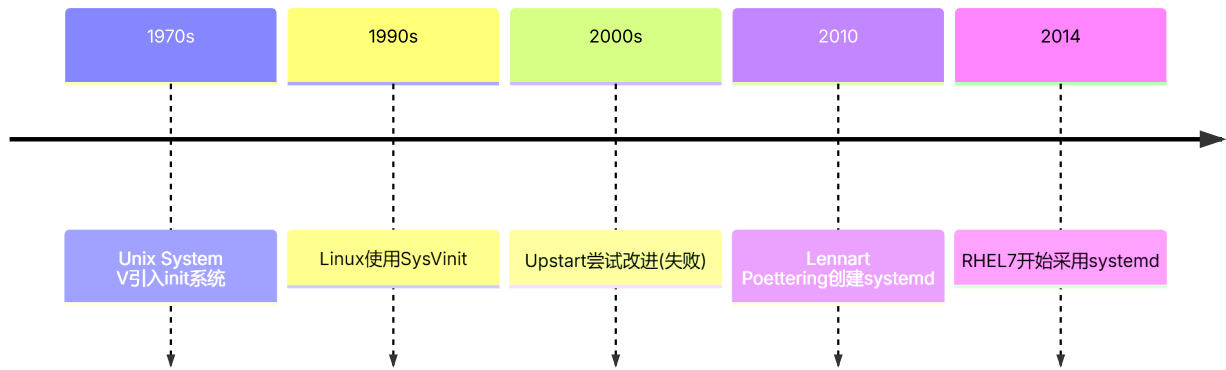
1. 系统启动：BIOS POST → Boot Loader (GRUB) → 内核加载
2. 运行级别切换：读取 `/etc/inittab` 确定运行级别（如3表示多用户文本模式）
3. 串行启动问题：执行 `/etc/rc.d/rc.3` 脚本，按顺序启动 `S*` 开头的服务
 - 服务必须等待前一个完成后才能启动下一个
 - 即使服务间无依赖关系，也必须串行执行
4. 典型启动时间：60-120秒（取决于服务数量）

核心问题：

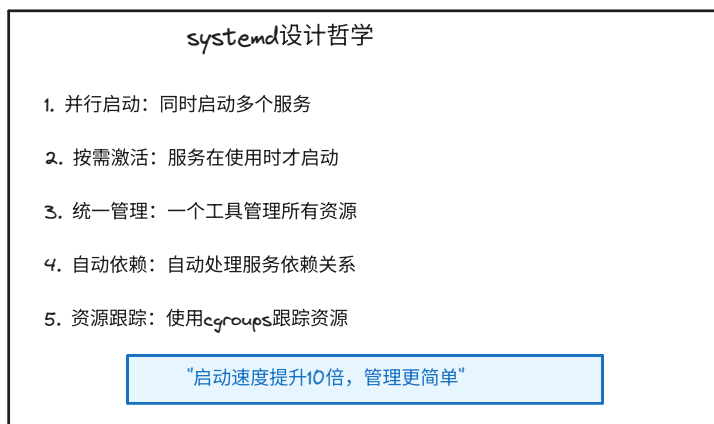
- 无法利用多核CPU并行优势
- 热插拔设备需要重新启动脚本
- 无法跟踪服务的资源使用

2010年：systemd的诞生

Linux初始化系统发展史



systemd的革命性设计：



设计优势说明：

1. 并行启动：多个独立服务可同时启动，充分利用多核CPU
2. 按需激活：Socket/D-Bus激活机制，服务仅在真正需要时启动
3. 统一管理：**systemctl** 一个命令管理所有系统资源
4. 自动依赖：自动解析服务间依赖关系，无需手动排序
5. 资源跟踪：使用cgroups精确控制每个服务的资源使用

性能对比：

SysVinit: 60-120秒 (服务越多越慢)
systemd: 10-15秒 (几乎不受服务数量影响)

为什么RHEL选择systemd?

1. 启动速度：并行启动使启动时间从1分钟降至10秒
2. 功能强大：集成日志管理（journald）、网络管理（networkd）
3. 易于管理：统一命令systemctl管理所有系统资源
4. 现代化：支持容器、快照等现代特性

⚠ 重要性：

- systemd是服务管理核心，必须掌握
- systemctl命令是系统管理的基础

第一部分：systemd和服务管理

1.1 systemd架构深度解析

什么是systemd？

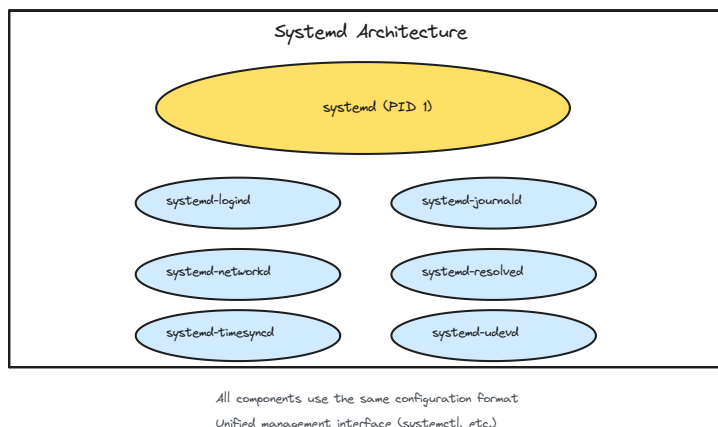
systemd是Linux系统的初始化系统和服务管理器，PID始终为1，是系统启动后的第一个进程。

systemd的核心特性：

1. PID为1 - 系统的第一个进程
2. 并行启动 - 同时启动多个独立服务
3. 按需激活 - 服务在使用时才启动
4. 依赖管理 - 自动处理服务依赖关系
5. 资源跟踪 - 使用cgroups跟踪资源使用
6. 快照支持 - 可以保存和恢复系统状态

SHELL

systemd的架构：



架构组件说明：

核心层：

- **systemd (PID=1)**：系统第一个进程，管理所有单元和系统管理器

系统管理器（各司其职）：

- **journald**：集中化日志管理（替代syslog）
- **logind**：用户登录会话管理
- **networkd**：基础网络配置（可选，通常用NetworkManager）
- **timedated**：系统时间和时区管理
- **hostnamed**：主机名管理
- **resolved**：DNS和mDNS解析（可选）

systemd单元（Units）详解：

systemd使用"单元"（Unit）来管理各种系统资源。每个单元都有一个配置文件和特定的类型。

单元类型	后缀	说明	示例	重要性
Service	.service	系统服务	sshd.service	☆☆☆
Socket	.socket	IPC套接字	sshd.socket	☆☆
Target	.target	同步点（类似运行级别）	multi-user.target	☆☆☆
Mount	.mount	文件系统挂载点	/home.mount	☆☆
Automount	.automount	自动挂载点	/data.automount	☆
Timer	.timer	定时器（类似cron）	backup.timer	☆☆
Path	.path	路径监控	print.path	☆
Slice	.slice	资源限制组	system.slice	☆
Scope	.scope	外部创建的进程	-	☆

⚠️ 重点：Target单元

```
# Target是systemd中的"同步点"
```

```
# 类似于SysVinit的"运行级别"
```

常用Target:

```
poweroff.target      = 关机 (运行级别0)
rescue.target        = 单用户模式 (运行级别1)
multi-user.target    = 多用户文本模式 (运行级别3)
graphical.target     = 图形界面模式 (运行级别5)
reboot.target        = 重启 (运行级别6)
```

```
# 查看当前target
```

```
systemctl get-default
```

```
# 输出: multi-user.target
```

```
# 设置默认target
```

```
systemctl set-default graphical.target
```

systemd配置文件位置:

systemd使用三层配置文件结构:

第1层: `/usr/lib/systemd/system/`

- 系统默认配置
- 不要修改!

第2层: `/etc/systemd/system/`

- 系统管理员配置
- 可以修改

第3层: `/run/systemd/system/`

- 运行时配置
- 临时性配置

优先级规则: $3 > 2 > 1$

/etc中的配置会覆盖/usr中的配置

⚠ 警告: 配置文件修改位置

× 错误做法:

```
vim /usr/lib/systemd/system/sshd.service
```

→ 系统更新后会覆盖你的修改!

✓ 正确做法:

1. 使用systemctl edit创建覆盖配置

```
systemctl edit sshd.service
```

在/etc/systemd/system/sshd.service.d/override.conf中编辑

2. 或直接复制配置文件到/etc

```
cp /usr/lib/systemd/system/sshd.service /etc/systemd/system/
```

```
vim /etc/systemd/system/sshd.service
```

练习名称	对应章节	说明
练习：识别自动启动的系统进程	第09章 第01节	识别systemd管理的服务

1.2 systemctl命令完全指南

systemctl - 查看服务:

列出所有active状态的service

```
systemctl list-units --type=service
```

列出所有service(包括inactive)

```
systemctl list-units --type=service --all
```

输出说明:

```
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
cron.service                       loaded active running Regular background
program processing daemon
```

字段详解:

字段	说明	可能的值
UNIT	服务单元名称	<code>sshd.service</code> , <code>cron.service</code>
LOAD	配置是否已加载	<code>loaded</code> (已加载), <code>not-found</code> (未找到), <code>error</code> (错误)
ACTIVE	高级激活状态	<code>active</code> (活动), <code>inactive</code> (未活动), <code>failed</code> (失败)
SUB	低级激活状态	<code>running</code> (运行中), <code>dead</code> (停止), <code>exited</code> (已退出)
DESCRIPTION	服务描述	服务的功能说明

LOAD 状态值：

- `loaded`：配置文件已成功加载
- `not-found`：找不到配置文件
- `error`：配置文件有语法错误
- `masked`：服务被屏蔽（无法启动）

ACTIVE + SUB 组合含义：

```
active (running)  → 服务正在运行
active (exited)   → 一次性任务已完成
inactive (dead)   → 服务未运行
failed (dead)     → 服务启动失败
active (waiting)  → 服务正在等待某事件
```

查看特定服务状态：


```
# 查看ssh服务状态
systemctl status sshd

# 输出示例:
# • sshd.service - OpenSSH server daemon
#   Loaded: loaded (/usr/lib/systemd/system/sshd.service;
#   enabled; vendor preset: enabled)
#   Active: active (running) since Mon 2025-01-15 10:00:00 CST;
#   5min ago
#   Docs: man:sshd(8) man:sshd_config(5)
#   Main PID: 1234 (sshd)
#   Tasks: 1 (limit: 4584)
#   Memory: 2.5M
#   CGroup: /system.slice/sshd.service
#           └─1234 /usr/sbin/sshd -D -oCiphers=aes256-
#             gcm@openssh.com,chacha20-poly1305@openssh.com
```

输出详解:

行	字段	说明
Loaded	配置状态	配置文件路径、是否开机启动、厂商默认
Active	运行状态	当前状态、启动时间
Docs	文档链接	相关手册页
Main PID	主进程ID	服务的主进程号
Tasks	任务数	当前任务数量（系统限制）
Memory	内存使用	当前内存占用
CGroup	控制组	进程所属的cgroup

Enabled vs Disabled:

- enabled → 开机自动启动（创建符号链接）
- disabled → 开机不启动（删除符号链接）
- static → 无法开机启动，但可手动启动
- indirect → 被其他服务依赖时启动
- masked → 被屏蔽，完全无法启动（链接到/dev/null）

Vendor Preset:

- 厂商默认设置（RHL9官方配置）

- 可以被管理员配置覆盖

快速检查服务状态：

```
# 检查服务是否正在运行
systemctl is-active sshd

# 检查服务是否开机启动
systemctl is-enabled sshd

# 检查服务是否启动失败
systemctl is-failed sshd

# 列出所有失败的服务
systemctl --failed --type=service
```

SHELL

1.3 管理服务

启动和停止服务：

```
# 启动服务
systemctl start sshd

# 停止服务
systemctl stop sshd

# 重启服务
systemctl restart sshd

# 重新加载配置(不中断服务)
systemctl reload sshd
```

SHELL

设置开机启动：

```
# 设置服务开机启动
systemctl enable sshd

# 禁止服务开机启动
systemctl disable sshd

# 启动并设置开机启动
systemctl enable --now sshd

# enable --now 相当于以下两个命令的组合：
systemctl enable sshd
systemctl start sshd
```

屏蔽服务：

```
# 屏蔽服务(防止启动)
systemctl mask sendmail

# 取消屏蔽
systemctl unmask sendmail

# 说明：
# mask = 创建符号链接到/dev/null, 无法启动
# disabled = 不开机启动, 但可以手动启动
```

停止服务, 同时服务自动运行, 以 firewalld 服务为例

```
systemctl stop firewalld
systemctl disable firewalld
```

查看依赖关系：

```
# 查看服务的依赖
systemctl list-dependencies sshd.service

# 查看反向依赖
systemctl list-dependencies sshd.service --reverse
```

练习名称	对应章节	说明
练习：CONTROLLING SYSTEM SERVICES	第09章 第02节	控制系统服务
总练习：CONTROLLING SERVICES AND DAEMONS	第09章 P310	服务和守护进程综合练习

第二部分：SSH完全指南

课前故事：SSH取代Telnet的革命

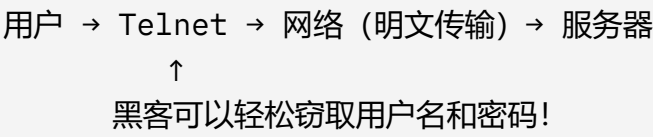
1980-1990年代：Telnet时代

在SSH出现之前，网络管理员使用**Telnet**进行远程登录：

Telnet的问题：

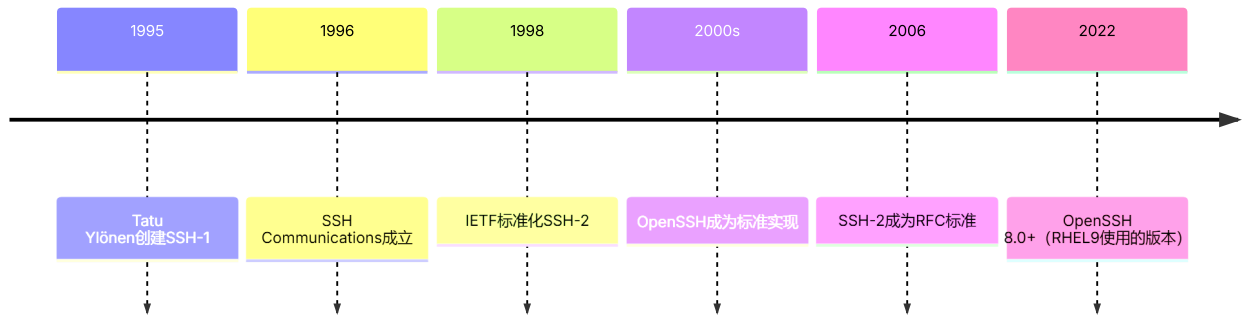
- 1. 明文传输 - 用户名和密码以明文传输
- 2. 无加密 - 所有数据都可以被窃听
- 3. 无认证 - 容易遭受中间人攻击

窃听场景：

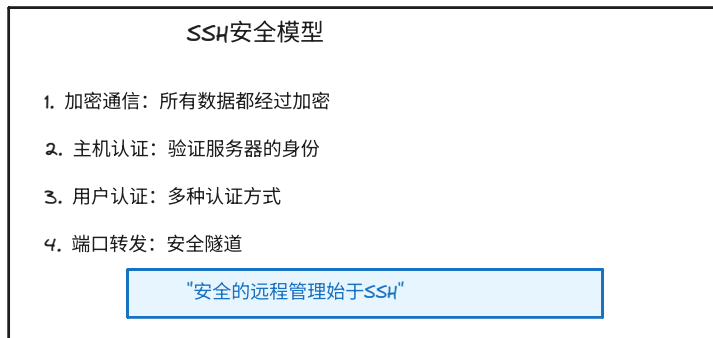


1995年：SSH的诞生

SSH发展简史



SSH的革命性设计：



安全特性详解：

1. 加密通信：

- 所有数据（包括用户名、密码、命令输出）都通过加密通道传输
- 使用对称加密（AES、ChaCha20）加密数据
- 即使网络被窃听，黑客也只能看到密文

2. 主机认证：

- 服务器向客户端发送主机公钥指纹
- 客户端验证指纹，确保连接到正确的服务器
- 防止中间人攻击（MITM）

3. 用户认证：

- 支持多种认证方式：密钥认证、密码认证、键盘交互
- 密钥认证比密码更安全、更方便

4. 端口转发：

- 通过SSH加密隧道转发任意TCP连接

- 可安全地访问远程服务

SSH vs Telnet对比:

特性	Telnet	SSH	重要性
加密	✗ 明文	✓ 加密	☆☆☆
认证	✗ 无	✓ 主机+用户认证	☆☆☆
端口转发	✗ 无	✓ 支持	☆☆
安全性	✗ 极低	✓ 高	☆☆☆

⚠ 重要性:

- SSH是RHCSA的必备技能
- SSH密钥认证是常见题目
- SSH配置文件修改可能考到

2.1 SSH连接完全指南

SSH (Secure Shell) 基础语法:

```
ssh [选项] 用户@主机
```

SHELL

基本连接

```
ssh user@hostname
```

示例:

```
ssh student@servera
```

```
ssh root@192.168.1.10
```

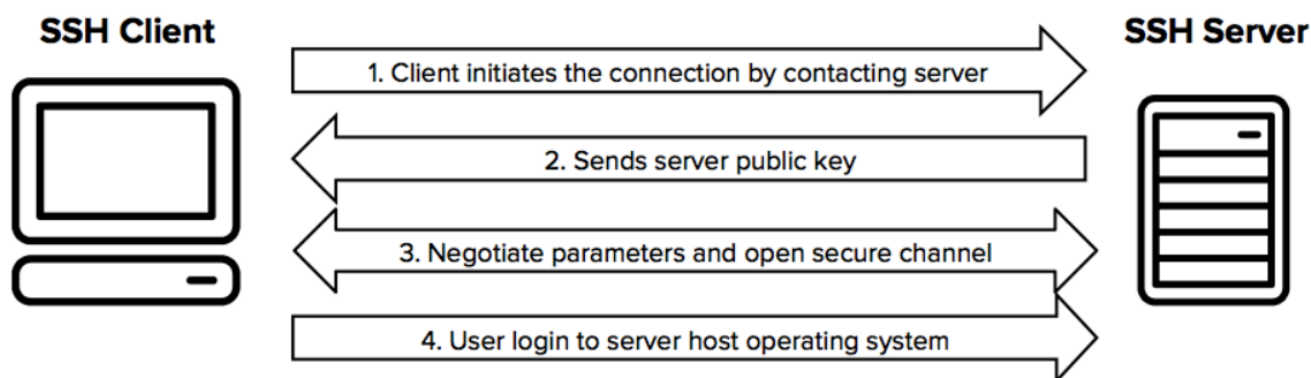
参数说明:

student = 远程系统的用户名

servera = 远程主机名或IP地址

SSH连接流程详解:

1. 客户端发起连接：用户在本地计算机上运行SSH客户端，指定要连接的远程服务器的地址。
2. 服务器响应：服务器接收到连接请求后，会发送其公钥给客户端。
3. 验证服务器：客户端会询问用户是否信任这个服务器的公钥。如果用户信任，客户端会将服务器的公钥(只和主机关联,和用户无关)保存在当前用户的 `known_hosts` 文件中。
4. 用户认证：
 - 密码认证：用户输入用户名和密码。服务器会验证密码是否正确。
 - 公钥/私钥认证：用户拥有一对密钥（公钥和私钥）。公钥已经存储在服务器上，私钥保留在客户端。客户端使用私钥对数据进行签名，服务器使用公钥验证签名。
5. 建立会话：一旦身份验证成功，客户端和服务端将建立一个加密的会话。通信使用对称加密, 因为对称加密方式在加密解密信息更高效



公钥和私钥的关系：
门上的锁头是公钥, 兜里的钥匙是私钥
古代的将军携带的虎符道理类似

如果要为用户生成密钥对:

`ssh-keygen` 命令可以为用户创建公钥 644 和私钥 600

免密登录: `ssh-copyid 主机` 分享公钥后可以实现免密登录到制定主机


```
# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# The default is to check both .ssh/authorized_keys and .ssh/authorized_keys2
# but this is overridden so installations will only check .ssh/authorized_keys
AuthorizedKeysFile /etc/ssh/authorized_keys .ssh/authorized_keys

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
```

首次连接示例:

```
# 首次连接到servera
ssh student@servera
```

SHELL

```
# 输出示例:
```

```
The authenticity of host 'servera (192.168.1.10)' can't be
established.
```

```
ECDSA key fingerprint is SHA256:abc123...
```

```
Are you sure you want to continue connecting (yes/no)? yes
```

```
# ↑ 首次连接需要确认主机指纹
```

```
Warning: Permanently added 'servera,192.168.1.10' (ECDSA) to the
list of known hosts.
```

```
# ↑ 指纹已保存到 ~/.ssh/known_hosts
```

```
student@servera's password:
```

```
# ↑ 输入密码
```

```
[student@servera ~]$
```

```
# ↑ 成功登录!
```

 警告: 主机指纹验证

```
# 如果看到以下警告：
WARNING: REMOTE HOST IDENTIFICATION CHANGED!
# → 主机指纹已改变！可能存在中间人攻击！
# → 或者服务器重新安装了系统

# 解决方法：
# 1. 如果确认服务器重新安装，删除旧指纹：
ssh-keygen -R servera

# 2. 或者手动编辑：
vim ~/.ssh/known_hosts
# 删除包含servera的行
```

SSH常用选项：

选项	说明	示例	重要性
-p	指定端口	ssh -p 2222 user@host	☆☆☆
-i	指定私钥	ssh -i key.pem user@host	☆☆
-v	详细模式（调试）	ssh -v user@host	☆
-vvv	最详细模式	ssh -vvv user@host	☆
-X	启用X11转发	ssh -X user@host	☆
-Y	启用可信X11转发	ssh -Y user@host	☆
-L	本地端口转发	ssh -L 8080:localhost:80 user@host	☆☆
-R	远程端口转发	ssh -R 8080:localhost:80 user@host	☆☆
-t	强制分配伪终端	ssh -t user@host command	☆☆

指定端口连接（常见）：

```
# SSH默认端口: 22
# 如果服务器使用非标准端口, 需要指定

# 语法:
ssh -p 端口号 用户@主机

# 示例1: 连接到端口2222
ssh -p 2222 student@servera

# 示例2: 连接到IP地址的特定端口
ssh -p 2022 root@192.168.1.100

# 示例3: 同时使用私钥文件
ssh -i ~/.ssh/mykey -p 2222 student@servera
```

在远程执行命令（不登录）：

```
# 语法:
ssh user@hostname "命令"

# 示例1: 查看远程主机名
ssh student@servera "hostname"
# 输出: servera.lab.example.com

# 示例2: 查看远程系统信息
ssh root@servera "uname -r"
# 输出: 5.14.0-...

# 示例3: 执行多个命令
ssh student@servera "hostname && ip addr show"
# 使用&&或;连接多个命令

# 示例4: 使用管道
ssh student@servera "cat /etc/passwd | grep root"

# 示例5: 在远程执行脚本
ssh student@servera "bash /path/to/script.sh"
```

⚠ 技巧：远程执行命令的引号

☒ 正确：使用双引号（可以展开本地变量）

```
ssh user@host "echo $PATH"
```

☒ 正确：使用单引号（在远程展开变量）

```
ssh user@host 'echo $PATH'
```

X 错误：忘记引号

```
ssh user@host echo $PATH
```

→ 会在本地执行echo，而不是远程执行！

示例对比：

```
ssh user@host "echo $HOME"    # 显示本地用户的HOME
```

```
ssh user@host 'echo $HOME'    # 显示远程用户的HOME
```

使用SSH配置文件（`~/.ssh/config`）：

```
# 配置文件路径: ~/.ssh/config

# 示例配置文件:
Host servera
    HostName 192.168.1.10
    User student
    Port 22

Host webserver
    HostName www.example.com
    User admin
    Port 2222
    IdentityFile ~/.ssh/web_key

Host *
    User root
    Port 22

# 使用配置文件后:
# 简化前: ssh -p 2222 -i ~/.ssh/web_key admin@www.example.com
# 简化后: ssh webserver

# 创建配置文件:
vim ~/.ssh/config
# 添加上述内容

# 设置权限 (必须! )
chmod 600 ~/.ssh/config
```

⚠ SSH配置文件权限警告

```
# SSH配置文件要求严格的权限:
# ~/.ssh/config      必须是 600 或 400
# ~/.ssh/id_rsa      必须是 600
# ~/.ssh/id_rsa.pub  必须是 644

# 如果权限太宽松, SSH会拒绝使用:
# Bad permissions 'rw-rw-rw-' for '/home/user/.ssh/config'

# 修复权限:
chmod 600 ~/.ssh/config
chmod 600 ~/.ssh/id_rsa
chmod 644 ~/.ssh/id_rsa.pub
```

2.2 SSH密钥认证完全指南

SSH密钥认证的优势：

SHELL

传统密码认证的问题：

1. 需要记忆密码

2. 可能被暴力破解

3. 可能被键盘记录器窃取

4. 无法实现自动化

SSH密钥认证的优势：

1. 无需输入密码

2. 更安全（2048位或更长密钥）

3. 可以实现自动化

4. 适合脚本和备份任务

SSH密钥对工作原理：



密钥认证流程说明：

1. 密钥生成：使用 `ssh-keygen` 生成公钥和私钥对
- 私钥保存在客户端 `~/.ssh/id_rsa`（必须保密）

• 公钥复制到服务器 `~/.ssh/authorized_keys`
2. 登录请求：
- 客户端发起SSH连接请求

• 服务器检查用户的 `authorized_keys` 文件
3. 质询响应：

- 服务器发送随机质询
- 客户端用私钥对质询签名
- 服务器用公钥验证签名

4. 认证完成：验证通过后直接登录，无需密码

安全性说明：

- 私钥永远不离开客户端
- 每次登录使用不同的质询，防止重放攻击
- 即使私钥文件被窃取，没有密码短语也无法使用

密钥类型对比：

算法	密钥长度	安全性	速度	兼容性	推荐
RSA	2048/4096位	高	慢	最好	★ ★ ★
ECDSA	256/384位	高	快	好	★ ★
Ed25519	256位	极高	极快	较新	★ ★ ★

生成SSH密钥对：

基本语法

`ssh-keygen -t 类型 -b 位数`

示例1: 生成RSA密钥 (推荐)

`ssh-keygen -t rsa -b 4096`

交互过程:

Generating public/private rsa key pair.

Enter file in which to save the key (/root/.ssh/id_rsa):

按Enter使用默认路径

Enter passphrase (empty for no passphrase):

输入密码短语 (可选, 增加安全性)

Enter same passphrase again:

再次输入密码短语

输出示例:

Your identification has been saved in /root/.ssh/id_rsa

Your public key has been saved in /root/.ssh/id_rsa.pub

The key fingerprint is:

SHA256:abc123... root@server1

The key's randomart image is:

+---[RSA 4096]-----+

| .o. |

...

+-----[SHA256]-----+

⚠ 密码短语 (Passphrase) vs 密码 (Password)

密码短语 (Passphrase) :

- 用于保护私钥文件

- 可以是任意长度的短语

- 可选设置 (通常留空)

建议:

生产环境: 设置密码短语

实际环境: 留空 (方便使用)

自动化脚本: 留空 (无法输入密码)

生成的文件说明:


```
# 密钥文件位置:
~/.ssh/id_rsa      # 私钥 (必须保密!)
~/.ssh/id_rsa.pub  # 公钥 (可以公开)

# 查看私钥:
cat ~/.ssh/id_rsa
# -----BEGIN OPENSSH PRIVATE KEY-----
# b3BlbnNzaC1rZXktdjEAAAA...
# -----END OPENSSH PRIVATE KEY-----

# 查看公钥:
cat ~/.ssh/id_rsa.pub
# ssh-rsa AAAAB3NzaC1yc2E... root@server1

# ⚠ 安全警告:
# - 永远不要分享私钥 (id_rsa)
# - 公钥 (id_rsa.pub) 可以自由分享
# - 私钥文件权限必须是600
```

复制公钥到服务器:

```
# 方法1: 使用ssh-copy-id (最简单, 推荐)

# 基本语法
ssh-copy-id user@hostname

# 示例1: 复制公钥到servera
ssh-copy-id student@servera

# 交互过程:
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/root/.ssh/id_rsa.pub"
student@servera's password:
# ↑ 输入一次密码 (最后一次! )

Number of key(s) added: 1

# 示例2: 指定端口
ssh-copy-id -p 2222 student@servera

# 示例3: 指定公钥文件
ssh-copy-id -i ~/.ssh/mykey.pub student@servera

# 方法2: 手动复制 (备用方法)

# 步骤1: 查看公钥内容
cat ~/.ssh/id_rsa.pub

# 步骤2: 在服务器上创建目录
ssh student@servera "mkdir -p ~/.ssh && chmod 700 ~/.ssh"

# 步骤3: 添加公钥到authorized_keys
ssh student@servera "echo 'ssh-rsa AAAA...' >>
~/.ssh/authorized_keys"

# 步骤4: 设置正确的权限
ssh student@servera "chmod 600 ~/.ssh/authorized_keys"
```

 易错点: **authorized_keys**权限

```
# SSH对authorized_keys文件权限要求严格:
# ~/.ssh/           必须是 700 (drwx-----)
# ~/.ssh/authorized_keys 必须是 600 (-rw-----)

# 如果权限不对, SSH会拒绝连接!
# Permission denied (publickey,gssapi-keyex,gssapi-with-mic)

# 修复权限:
ssh user@server "mkdir -p ~/.ssh && chmod 700 ~/.ssh"
ssh user@server "chmod 600 ~/.ssh/authorized_keys"
```

测试密钥登录:

```
# 测试1: 基本连接测试
ssh student@servera
# 成功: 直接登录, 无需输入密码!

# 测试2: 使用-v选项查看认证过程
ssh -v student@servera
# 输出会显示详细的认证过程
# debug1: Offering public key: /root/.ssh/id_rsa RSA SHA256:...
# debug1: Server accepts key: pkalg rsa-sha2-512 blen 256
# debug1: Authentication succeeded (publickey).

# 测试3: 验证私钥权限
ls -l ~/.ssh/id_rsa
# -rw-----. 1 root root 1.2K Jan 15 10:00 id_rsa
#          ↑
#          必须是 600 (rw-----)

# 测试4: 验证公钥已添加到服务器
ssh student@servera "cat ~/.ssh/authorized_keys"
# 应该能看到你的公钥内容
```

⚠ 实际场景: 密钥认证失败排查

```
# 问题：密钥认证失败，仍然要求输入密码

# 排查步骤1：验证私钥存在
ls -l ~/.ssh/id_rsa

# 排查步骤2：验证私钥权限
stat -c %a ~/.ssh/id_rsa
# 应该显示：600

# 排查步骤3：验证公钥已添加
ssh student@servera "cat ~/.ssh/authorized_keys"

# 排查步骤4：检查SELinux（可能影响）
getenforce
# 如果是Enforcing, 检查上下文：
ls -Z ~/.ssh/authorized_keys

# 排查步骤5：查看详细日志
ssh -vvv student@servera
```

练习名称	对应章节	说明
练习：ACCESSING THE REMOTE COMMAND LINE	第10章 第01节	SSH远程登录基础
练习：CONFIGURING SSH KEY-BASED AUTHENTICATION	第10章 第02节	配置SSH密钥认证

2.3 SSH服务配置 (sshd_config)

SSH配置文件位置：



配置文件说明：

1. 客户端配置 `/etc/ssh/ssh_config`:

- 控制SSH客户端行为
- 定义连接默认参数
- 一般考试不需要修改

2. 服务器配置 `/etc/ssh/sshd_config`:

- 控制SSH服务器行为（考试重点）
- 定义端口、认证方式、用户限制等
- 修改后需执行 `systemctl reload sshd`

常见SSH服务器配置选项:

配置项	默认值	说明	重要性
Port	22	SSH监听端口	☆☆☆
PermitRootLogin	yes	是否允许root登录	☆☆☆
PasswordAuthentication	yes	是否允许密码认证	☆☆☆
PubkeyAuthentication	yes	是否允许密钥认证	☆☆
AllowUsers	-	仅允许特定用户登录	☆☆
DenyUsers	-	拒绝特定用户登录	☆☆
X11Forwarding	no	是否允许X11转发	☆

修改SSH配置文件（实际场景）：

```
# 场景1: 禁用root SSH登录

# 步骤1: 备份配置文件
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak

# 步骤2: 编辑配置文件
vim /etc/ssh/sshd_config

# 查找并修改:
# PermitRootLogin yes
# 改为:
# PermitRootLogin no

# 或者添加注释和取消注释:
# #PermitRootLogin yes
# PermitRootLogin no

# 步骤3: 验证配置文件语法
sshd -t
# 无输出表示语法正确

# 步骤4: 重新加载SSH服务
systemctl reload sshd

# 步骤5: 验证配置
grep PermitRootLogin /etc/ssh/sshd_config
```

⚠ 警告: reload vs restart

```
# systemctl reload sshd
# - 重新加载配置
# - 不中断现有SSH连接
# - 配置错误会导致服务无法重新加载

# systemctl restart sshd
# - 重启服务
# - 中断现有SSH连接
# - 配置错误会导致服务启动失败

# 推荐: 使用reload
# 如果修改了端口, 使用restart
```

配置示例（常见场景）：

```
# 场景1: 修改SSH端口为2222
vim /etc/ssh/sshd_config
# Port 22
# 改为:
# Port 2222

systemctl restart sshd
# 注意: 修改端口后必须重启服务

# 验证:
ss -tlnp | grep sshd
# 应该显示: *:2222

# 场景2: 仅允许特定用户登录
vim /etc/ssh/sshd_config
# 添加:
# AllowUsers student admin
# → 只有student和admin可以登录

# 或拒绝特定用户:
# DenyUsers testuser
# → testuser无法登录

systemctl reload sshd

# 场景3: 禁用密码认证, 强制使用密钥
vim /etc/ssh/sshd_config
# PasswordAuthentication no
# PubkeyAuthentication yes

systemctl reload sshd

# ⚠ 警告: 在确保密钥认证工作之前, 不要禁用密码认证!
# 否则可能把自己锁在门外!
```

⚠ 必会: 修改SSH配置的标准流程

```
# 标准流程:

# 1. 备份配置文件
cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup

# 2. 编辑配置文件
vim /etc/ssh/sshd_config
# 修改需要的配置项

# 3. 验证配置语法
sshd -t

# 4. 重新加载服务
systemctl reload sshd

# 5. 验证配置生效
grep "修改的配置项" /etc/ssh/sshd_config
```

练习名称	对应章节	说明
练习：CUSTOMIZING OPENSSSH SERVICE CONFIGURATION	第10章 第03节	自定义SSH服务配置

2.4 SSH故障排查

SSH连接故障排查流程：

SSH连接失败排查步骤

SSH连接失败排查步骤 ^title

1. 检查网络连通性

ping host

2. 检查SSH端口是否开放

telnet host 22 nc -zv host 22

3. 检查SSH服务状态

systemctl status sshd

4. 检查防火墙

firewall-cmd --list-all

5. 使用详细模式连接

ssh -vvv user@host

排查步骤说明：

1. 网络连通性 (`ping host`): 验证基础网络连接是否正常
2. 端口检查 (`nc -zv host 22` 或 `telnet host 22`): 确认SSH端口22是否开放
3. 服务状态 (`systemctl status sshd`): 检查SSH服务是否运行
4. 防火墙 (`firewall-cmd --list-all`): 确认防火墙规则允许SSH
5. 详细日志 (`ssh -vvv user@host`): 使用最详细模式获取调试信息

常见SSH问题及解决:

SHELL

```
# 问题1: Connection refused
# 原因1: SSH服务未启动
systemctl status sshd
systemctl start sshd

# 原因2: 防火墙阻止
firewall-cmd --add-service=ssh --permanent
firewall-cmd --reload

# 问题2: Connection timed out
# 原因: 网络不通或防火墙阻止
ping host
telnet host 22

# 问题3: Permission denied (publickey)
# 原因1: 公钥未添加
ssh-copy-id user@host

# 原因2: 密钥权限错误
chmod 600 ~/.ssh/id_rsa

# 原因3: authorized_keys权限错误
ssh user@host "chmod 600 ~/.ssh/authorized_keys"

# 问题4: Host key verification failed
# 原因: 主机指纹改变
ssh-keygen -R host
```

练习名称	对应章节	说明
练习: 配置和保护SSH	第10章 P344	开放研究试验

课前故事：网络配置的演变

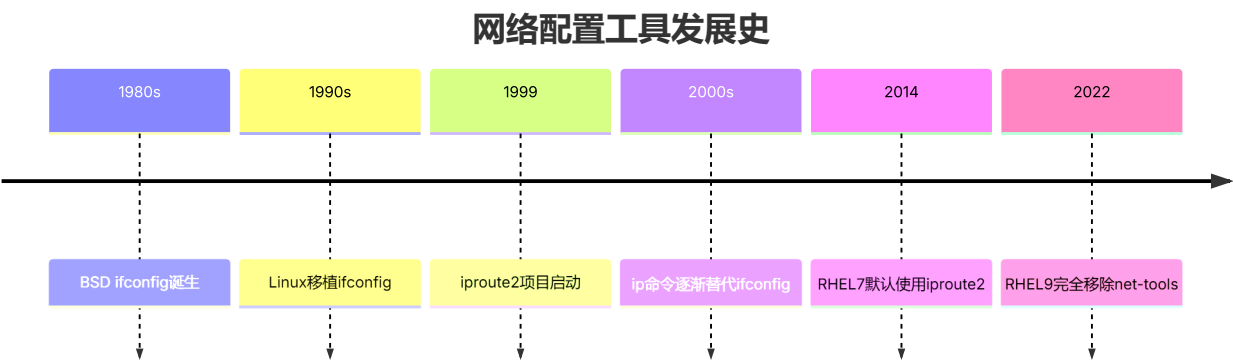
1990年代：ifconfig时代

在Linux早期，网络配置使用的是**ifconfig**命令，属于net-tools软件包：

```
传统网络配置工具 (net-tools) :  
ifconfig eth0 192.168.1.10 netmask 255.255.255.0  
route add default gw 192.168.1.1
```

- 问题：
- 1. 不支持IPv6
 - 2. 输出格式不统一
 - 3. 无法处理复杂网络场景
 - 4. 已停止维护（2001年）

2000年代：iproute2的诞生



- **ifconfig** 命令仍然可用, 属于 **net-tools** 包, 需要单独安装
- **RHEL7+** 开始网络服务从 network 升级为 NetworkManager
- nmcli是NetworkManager的主要工具, 提供了强大的 **nmcli** 和 **nmtui** 工具, 以及图形化配置界面。
- 网络配置是RHCSA考试的重要内容

第三部分：网络管理概述

本部分涵盖网络基础、网络配置、主机名和DNS配置等内容。

第四部分：网络基础

4.1 网络接口命名规则详解

RHEL7+网络接口命名（Predictable Network Interface Names）：

传统命名方式（eth0, eth1）存在不确定性：

- 问题场景：
- 多个网卡：哪个是eth0，哪个是eth1？
 - 添加硬件：eth0可能变成eth1！
 - 重启后：命名可能改变

新的命名规则（systemd预测性命名）：

前缀	命名方案	说明	示例	重要性
eno	Onboard	板载设备（索引）	eno1, eno2	☆☆
ens	Slot	PCI热插拔插槽（索引）	ens3, ens4	☆☆☆
enp	PCI	PCI设备位置（总线:设备）	enp0s3, enp1s0	☆☆☆
enx	MAC	基于MAC地址	enx78e7d1ea46da	☆
wlp	Wireless PCI	无线PCI设备	wlp4s0	☆
wls	Wireless USB	无线USB设备	wls1	☆

命名详解：

```
# enp0s3 解析:
en   = Ethernet (以太网)
p    = PCI bus
0    = Bus number 0
s    = Slot
3    = Slot number 3
# → 位于PCI总线0、插槽3的以太网设备

# ens4 解析:
en   = Ethernet (以太网)
s    = Slot
4    = Hot-plug slot index 4
# → 位于热插拔插槽4的以太网设备

# wlp4s0 解析:
wl   = Wireless LAN (无线局域网)
p    = PCI bus
4    = Bus number 4
s    = Slot
0    = Slot number 0
# → 位于PCI总线4、插槽0的无线设备
```

⚠ 实际场景：网络接口识别

```
# 题目：系统有两个网络接口，如何识别哪个连接到外部网络？

# 方法1：查看网络接口状态
ip link show
# 2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP>
#   link/ether 52:54:00:00:0a:brd ff:ff:ff:ff:ff:ff
#   ↑ UP = 接口已启动
# 3: ens4: <BROADCAST,MULTICAST,LOWER_UP>
#   link/ether 52:54:00:00:0b:brd ff:ff:ff:ff:ff:ff

# 方法2：查看IP地址
ip addr show
# ens3有192.168.1.10 (外部网络)
# ens4无IP或10.0.0.10 (内部网络)

# 方法3：查看路由
ip route show
# default via 192.168.1.1 dev ens3 # 默认路由在ens3
```

查看网络接口 (ip命令) :

SHELL

```
# 方法1: 查看所有网络接口
ip link show
# 或
ip a

# 输出示例:
1: lo: <LOOPBACK,UP,LOWER_UP>
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP>
    link/ether 52:54:00:00:0a brd ff:ff:ff:ff:ff:ff
3: ens4: <BROADCAST,MULTICAST,LOWER_UP>
    link/ether 52:54:00:00:0b brd ff:ff:ff:ff:ff:ff

# 方法2: 只看特定接口
ip link show ens3

# 方法3: 查看详细信息
ip -detail link show ens3

# 方法4: 查看接口统计信息
ip -s link show ens3
# 输出:
# 3: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP>
#      RX: bytes  packets  errors  dropped  overrun  mcast
#      1234567    4567      0        0         0         0
#      TX: bytes  packets  errors  dropped  carrier  collsns
#      234567     3456      0        0         0         0
```

接口标志解释:

标志	含义	说明	重要性
UP	接口已启用	可以收发数据	☆☆☆
LOWER_UP	链路已建立	物理连接正常	☆☆☆
BROADCAST	支持广播	以太网特性	☆☆
MULTICAST	支持组播	支持IGMP等	☆☆
LOOPBACK	回环接口	lo接口	☆☆
PROMISC	混杂模式	接收所有数据包	☆

⚠️ 重点：UP vs LOWER_UP

SHELL

```
# UP = 接口已启用（管理状态）
# LOWER_UP = 链路已建立（物理状态）

# 场景分析：
ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> # 正常，有连接
ens4: <BROADCAST,MULTICAST,UP>           # 接口启用但无物理连接
ens5: <BROADCAST,MULTICAST>               # 接口未启用
```

4.2 IP地址深度解析

IP地址基础：

SHELL

```
# IPv4地址组成：
IP地址 = 网络部分 + 主机部分（每段8位，共32位）

示例：192.168.1.10/24
    192.168.1.   = 网络部分
    10           = 主机部分
    /24          = 子网掩码位数（前24位是网络）
```

CIDR表示法（同时表示ip/子网掩码, 必记）：

CIDR	子网掩码	网络位	主机位	可用主机数	常见用途
/8	255.0.0.0	8	24	16,777,214	超大型网络（A级）
/16	255.255.0.0	16	16	65,534	大型网络（B级）
/24	255.255.255.0	24	8	254	标准局域网
/25	255.255.255.128	25	7	126	小型网络
/26	255.255.255.192	26	6	62	更小网络
/32	255.255.255.255	32	0	1	主机地址

⚠️ 技巧：快速计算可用主机数

公式：可用主机数 = $2^{(32-\text{掩码位数})} - 2$

-2的原因：网络地址和广播地址不能分配给主机

示例1：/24网络

可用主机数 = $2^{(32-24)} - 2 = 2^8 - 2 = 256 - 2 = 254$

示例2：/25网络

可用主机数 = $2^{(32-25)} - 2 = 2^7 - 2 = 128 - 2 = 126$

示例3：/26网络

可用主机数 = $2^{(32-26)} - 2 = 2^6 - 2 = 64 - 2 = 62$

示例4：/32网络

可用主机数 = $2^{(32-32)} - 2 = 2^0 - 2 = 1 - 2 = -1$

→ /32表示单个主机，不计算可用主机

查看IP地址（ip命令）：

查看所有IP地址

ip addr show

或

ip a

输出示例：

1: lo: <LOOPBACK,UP,LOWER_UP>

inet 127.0.0.1/8 scope host lo

2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP>

inet 192.168.1.10/24 brd 192.168.1.255 scope global dynamic

ens3

3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP>

inet 10.0.0.5/24 brd 10.0.0.255 scope global ens4

valid_lft forever preferred_lft forever

字段说明：

inet = IPv4地址

192.168.1.10 = IP地址

/24 = 子网掩码（CIDR表示）

brd = 广播地址

scope = 作用域

dynamic = 通过DHCP获取

valid_lft = 有效期

作用域（scope）类型：

```
# scope global - 全局地址, 可路由
# 示例: 192.168.1.10/24 scope global

# scope link - 链路本地地址, 不可路由
# 示例: 169.254.x.x/16 scope link (IPv4链路本地)

# scope host - 本机地址, 只在主机内有效
# 示例: 127.0.0.1/8 scope host
```

⚠ 警告: 链路本地地址 (169.254.0.0/16)

```
# 如果看到:
inet 169.254.x.x/16 scope link ens3
# → DHCP失败, 自动分配的链路本地地址
# → 无法连接到网络!
# → 需要检查DHCP服务器或手动配置IP
```

IPv6地址详解:

```
# IPv6地址格式:
2001:0db8:0000:0000:0000:0000:0000:0001

# 简写规则:
# 1. 省略前导零:
2001:db8:0:0:0:0:0:1

# 2. 连续的零用::代替 (只能一次):
2001:db8::1

# 示例对比:
完整:      2001:0db8:0000:0000:0000:0000:0000:0001
简化1:     2001:db8:0:0:0:0:0:1
简化2:     2001:db8::1

# IPv6前缀 (类似IPv4的子网掩码):
2001:db8::/32      # 前32位是网络
fe80::/10          # 链路本地地址
```

IPv6地址类型:

类型	前缀	说明	示例	重要性
链路本地	fe80::/10	本地网络，不可路由	fe80::1	☆☆☆
唯一本地	fc00::/7	类似IPv4私有地址	fc00::1	☆☆
全球单播	2000::/3	全球可路由地址	2001:db8::1	☆☆
环回地址	::1/128	本机环回	::1	☆☆☆

⚠️ 重点：IPv6环回地址

SHELL

```
# IPv4环回: 127.0.0.1
# IPv6环回: ::1

# 测试:
ping6 -c 1 ::1
# 或
ping -c 1 ::1 # RHEL9中ping同时支持IPv4和IPv6
```

练习名称	对应章节	说明
练习：VALIDATING NETWORK CONFIGURATION	第11章 第01节	验证网络配置

第五部分：nmcli网络配置完全指南

5.1 NetworkManager和nmcli

NetworkManager简介：

NetworkManager (NM) 是RHEL9的默认网络管理服务，提供：

- 动态网络配置
- WiFi管理
- VPN支持
- 与其他工具集成（nmcli, nmtui, GNOME设置）

架构说明：

NetworkManager采用分层架构设计：

- 用户接口层：提供多种管理方式
 - `nmcli`：命令行工具，适合脚本和SSH远程管理
 - `nmtui`：文本界面，适合交互式配置
 - GNOME设置：图形界面，适合桌面用户
- 核心层：NetworkManager守护进程
 - 管理所有网络连接的生命周期
 - 处理设备热插拔事件
 - 维护连接配置
- 插件层：支持不同网络类型
 - Ethernet插件：处理有线网络
 - WiFi插件：处理无线网络
 - PPPoE插件：处理宽带拨号
 - Team/Bond插件：处理链路聚合
- 后端层：与系统底层交互
 - `systemd-networkd`：配置内核网络接口
 - `udev`：检测设备变化
 - `dhclient`：获取DHCP地址
 - `iptables/nftables`：配置防火墙规则
- 配置文件：
 - `/etc/NetworkManager/system-connections/*.nmconnection`：现代格式
 - `/etc/sysconfig/network-scripts/ifcfg-*`：遗留格式（仍支持）

⚠ 重要性：nmcli是RHCSA的核心工具(可用nmtui代替)

```
# RHCSA可能要求：  
# 1. 配置静态IP地址  
# 2. 配置DHCP  
# 3. 配置DNS服务器  
# 4. 修改主机名
```

SHELL

nmcli基础语法：

```
nmcli [选项] 对象 {命令 | help}
```

对象类型:

device = 网络设备 (网卡)

connection = 连接配置 (配置文件)

常用命令:

nmcli device status # 查看设备状态

nmcli connection show # 查看所有连接

nmcli connection show ens3 # 查看特定连接详情

nmcli输出字段说明:

查看设备状态

```
nmcli device status
```

输出示例:

DEVICE	TYPE	STATE	CONNECTION
ens3	ethernet	connected	System ens3
ens4	ethernet	disconnected	--
wlp4s0	wifi	connected	MyWiFi

字段说明:

DEVICE = 设备名称 (ens3, ens4等)

TYPE = 设备类型 (ethernet, wifi, bridge等)

STATE = 设备状态

CONNECTION = 当前使用的连接配置

设备状态类型:

connected = 已连接

disconnected = 未连接

unavailable = 不可用 (如网线未插)

connecting = 正在连接

⚠ 设备 vs 连接 (重要概念!)

```
# 设备 (Device) = 物理或虚拟的网络接口
# 连接 (Connection) = 设备的配置文件

# 一个设备可以有多个连接配置，但只能使用一个
# 类比：
# 设备 = 网卡硬件
# 连接 = 网卡的配置文件 (IP、DNS等)

# 示例：
nmcli connection show
# NAME                UUID   TYPE      DEVICE
# System eth0         ...    ethernet  ens3
# My Static IP        ...    ethernet  --
# DHCP Connection     ...    ethernet  --
# → ens3设备当前使用"System eth0"连接
# → 其他两个连接配置存在但未激活
```

5.2 查看网络状态

查看设备状态（详细）：

方法1: 简单状态

```
nmcli dev status
```

方法2: 详细信息

```
nmcli device show ens3
```

输出示例:

```
GENERAL.DEVICE: ens3
GENERAL.TYPE: ethernet
GENERAL.HWADDR: 52:54:00:00:0a
GENERAL.MTU: 1500
GENERAL.STATE: 100 (connected)
GENERAL.CONNECTION: System ens3
GENERAL.CON-PATH: /org/freedesktop/NetworkManager/ActiveConnection/1
WIRED-PROPERTIES.CARRIER: on
IP4.ADDRESS[1]: 192.168.1.10/24
IP4.GATEWAY: 192.168.1.1
IP4.DNS[1]: 192.168.1.1
IP4.DOMAIN[1]: example.com
IP6.ADDRESS[1]: fe80::5054:ff:fe00:a/64
```

重要字段:

```
IP4.ADDRESS    = IPv4地址
IP4.GATEWAY    = 默认网关
IP4.DNS        = DNS服务器
IP4.DOMAIN     = DNS搜索域
```

查看连接配置:

查看所有连接

```
nmcli connection show
```

输出示例:

NAME	UUID	TYPE
DEVICE		
System ens3	320fb6b3-...	ethernet
ens3		
My Static IP	5b6f9b8d-...	ethernet
--		
Dhcp Connection	8c7f0e2e-...	ethernet
--		

字段说明:

NAME = 连接名称 (可自定义)
UUID = 唯一标识符 (自动生成)
TYPE = 连接类型
DEVICE = 当前使用的设备 (空表示未激活)

查看特定连接详情

```
nmcli connection show "System ens3"
```

输出示例:

connection.id:	System ens3
connection.uuid:	320fb6b3-...
connection.type:	802-3-ethernet
connection.autoconnect:	yes
ipv4.method:	auto
ipv4.addresses:	192.168.1.10/24
ipv4.gateway:	192.168.1.1
ipv4.dns:	192.168.1.1
ipv6.method:	auto

⚠ **ipv4.method (重要!)**

```
# ipv4.method决定IP地址配置方式:
# auto    = DHCP自动获取
# manual  = 手动静态配置
# shared  = 共享连接 (NAT)
# disabled = 禁用IPv4

# 查看当前配置:
nmcli -f ipv4.method connection show "System ens3"

# 修改为静态:
nmcli connection modify "System ens3" ipv4.method manual

# 修改为DHCP:
nmcli connection modify "System ens3" ipv4.method auto
```

5.3 配置静态IP地址 (必会)

场景：配置静态IP地址

```
# 场景：为ens3配置以下静态IP
# IP地址：192.168.1.100
# 子网掩码：/24
# 网关：192.168.1.1
# DNS：192.168.1.1, 8.8.8.8

# 步骤1：创建新连接或修改现有连接
nmcli connection modify "System ens3" \
    ipv4.method manual \
    ipv4.addresses 192.168.1.100/24 \
    ipv4.gateway 192.168.1.1 \
    ipv4.dns "192.168.1.1 8.8.8.8"

# 步骤2：重新加载配置
nmcli connection up "System ens3"

# 步骤3：验证配置
ip addr show ens3
# inet 192.168.1.100/24 brd 192.168.1.255 scope global ens3

ip route show
# default via 192.168.1.1 dev ens3

cat /etc/resolv.conf
# nameserver 192.168.1.1
# nameserver 8.8.8.8
```

⚠ 易错点：nmcli modify后的connection up

```
# ✗ 常见错误：
nmcli connection modify "System ens3" ipv4.addresses
192.168.1.100/24
# 修改后没有重新加载连接
# → 配置没有生效！

# ✔ 正确做法：
nmcli connection modify "System ens3" ipv4.addresses
192.168.1.100/24
nmcli connection up "System ens3" # 必须执行！
```

添加新连接（实际场景）：

场景：创建名为"office"的新连接，配置静态IP

步骤1：添加新连接

```
nmcli connection add \
  con-name office \
  type ethernet \
  ifname ens4 \
  ipv4.method manual \
  ipv4.addresses 10.0.0.100/24 \
  ipv4.gateway 10.0.0.1 \
  ipv4.dns 10.0.0.1 \
  autoconnect yes
```

参数说明：

con-name office = 连接名称

type ethernet = 连接类型

ifname ens4 = 网络接口

ipv4.method manual = 静态配置

autoconnect yes = 自动连接

步骤2：激活连接

```
nmcli connection up office
```

步骤3：验证

```
nmcli connection show office
```

```
ip addr show ens4
```

配置DHCP（实际场景）：

场景：配置ens3使用DHCP获取IP

步骤1：修改连接为DHCP

```
nmcli connection modify "System ens3" ipv4.method auto
```

步骤2：重新加载

```
nmcli connection up "System ens3"
```

步骤3：验证

```
ip addr show ens3
```

```
# inet 192.168.1.x/24 brd ... scope global dynamic ens3
```

```
#
```

↑

```
#
```

dynamic = DHCP获取

⚠️ DHCP陷阱：静态IP转DHCP

SHELL

场景：将静态IP改为DHCP

✗ 错误做法：

```
nmcli connection modify "System ens3" ipv4.method auto
```

静态IP配置仍然存在！

✅ 正确做法：

```
nmcli connection modify "System ens3" ipv4.method auto
```

```
nmcli connection modify "System ens3" ipv4.addresses ""
```

#

↑

#

清除静态地址

```
nmcli connection up "System ens3"
```

5.4 网络配置的持久化

NetworkManager的配置文件位置：

文件格式

- 每个连接对应一个 `*.nmconnection` 文件
- 使用INI格式配置

示例

```
/etc/NetworkManager/system-connections/  
├─ System ens3.nmconnection  
├─ office.nmconnection  
└─ home-wifi.nmconnection
```

特点

- 配置文件权限通常设置为 600（只有root可读写）
- 支持通过 `nmcli` 命令或直接编辑文件
- 手工修改后(非常不推荐)要用 `nmcli connection reload` 重新加载

查看配置文件：

查看连接的配置文件

```
nmcli connection show "System ens3" | grep connection.filename
```

输出:

connection.filename:

```
/etc/NetworkManager/system-connections/System ens3.nmconnection
```

查看配置文件内容

```
cat "/etc/NetworkManager/system-connections/System ens3.nmconnection"
```

输出示例:

```
[connection]
```

```
id=System ens3
```

```
uuid=320fb6b3-...
```

```
type=ethernet
```

```
autoconnect=true
```

```
[ipv4]
```

```
method=auto
```

```
address1=192.168.1.100/24,192.168.1.1
```

```
dns=192.168.1.1;8.8.8.8;
```

```
[ipv6]
```

```
method=auto
```

 **警告：不要手动编辑.nmconnection文件**

X 错误做法:

```
vim /etc/NetworkManager/system-connections/System ens3.nmconnection
```

→ 语法错误可能导致NetworkManager无法读取配置!

 正确做法:

使用nmcli修改配置

```
nmcli connection modify "System ens3" ...
```

练习名称	对应章节	说明
练习：CONFIGURING NETWORKING FROM THE COMMAND LINE	第11章 第02节	使用nmcli配置网络

练习名称	对应章节	说明
练习：EDITING NETWORK CONFIGURATION FILES	第11章 第03节	编辑网络配置文件

第六部分：主机名和DNS配置

6.1 主机名配置完全指南

主机名的三种类型：

SHELL

```
# 1. 静态主机名 (Static Hostname)
#   - 管理员配置的主机名
#   - 存储在/etc/hostname
#   - 系统重启后保持

# 2. 临时主机名 (Transient Hostname)
#   - 通过DHCP或mDNS获取
#   - 临时使用，重启后丢失

# 3. 漂亮主机名 (Pretty Hostname)
#   - 自由格式，可包含空格和特殊字符
#   - 仅用于显示，不用于系统标识
#   - 示例: "John's Desktop"
```

查看主机名：

```
# 方法1: 简单查看
hostname
# 输出: server1.example.com

# 方法2: 详细信息 (推荐)
hostnamectl status

# 输出示例:
#   Static hostname: server1.example.com
#           Icon name: computer-server
#           Chassis: server
#           Machine ID: 1234567890abcdef...
#           Boot ID: abcdef123456...
#   Virtualization: kvm
#   Operating System: Red Hat Enterprise Linux 9
#       CPE OS Name: cpe:/o:redhat:enterprise_linux:9::baseos
#           Kernel: Linux 5.14.0-...
#   Architecture: x86-64

# 重要字段:
#   Static hostname   = 静态主机名
#   Pretty hostname   = 漂亮主机名 (如果设置)
#   Icon name         = 图标名称 (用于显示)
#   Chassis           = 机箱类型
```

设置主机名 (必会) :

```
# 基本语法
hostnamectl set-hostname <新主机名>

# 示例1: 设置静态主机名
hostnamectl set-hostname server1.example.com

# 示例2: 同时设置静态和漂亮主机名
hostnamectl set-hostname server1.example.com \
    --pretty "Production Web Server 1"

# 示例3: 只设置漂亮主机名
hostnamectl set-hostname --pretty "Web Server 1"

# 验证修改
hostname
hostnamectl status
```

⚠ 易错点：主机名格式

SHELL

静态主机名规则：

- 只能包含字母、数字、连字符(-)和点(.)

- 不能以连字符开头或结尾

- 建议使用FQDN格式（完全限定域名）

✕ 错误示例：

hostnamectl set-hostname -server1 # 不能以-开头

hostnamectl set-hostname server_1 # 不能使用_

hostnamectl set-hostname My Server # 不能包含空格

✔ 正确示例：

hostnamectl set-hostname server1

hostnamectl set-hostname server1.example.com

hostnamectl set-hostname web-server-01.example.com

主机名解析 (/etc/hosts)：

```
# /etc/hosts的作用:
# 1. 本地主机名解析 (优先于DNS)
# 2. 离线系统的主机名映射
# 3. 加速常用地址的解析

# 查看hosts文件
cat /etc/hosts

# 默认内容:
127.0.0.1    localhost localhost.localdomain localhost4
localhost4.localhostdomain4
::1         localhost localhost.localdomain localhost6
localhost6.localhostdomain6

# 格式说明:
# IP地址      主机名      别名1 别名2
# 192.168.1.10 servera.example.com servera

# 添加自定义解析
vim /etc/hosts
# 添加:
192.168.1.100 server1.example.com server1
192.168.1.101 server2.example.com server2

# 测试解析
ping server1
# → 将使用192.168.1.100
```

⚠ 实际场景：修改/etc/hosts

```
# 场景：添加主机名解析
# server1.example.com → 192.168.1.100

# 步骤1：编辑文件
vim /etc/hosts

# 步骤2：添加行（文件末尾）
192.168.1.100 server1.example.com server1

# 步骤3：保存并测试
ping -c 1 server1
# PING server1 (192.168.1.100) ...
```

6.2 DNS配置完全指南

DNS解析顺序：

Linux系统按以下顺序解析域名：

第1步： `/etc/hosts`（静态解析）

- 本地主机表
- 优先级最高
- 立即返回结果

第2步：DNS服务器（动态解析）

- 查询 `/etc/resolv.conf` 中配置的DNS服务器
- 递归查询域名
- 返回解析结果

第3步：返回错误

- 如果DNS服务器也无法解析
- 返回 "Name or service not known" 错误

配置文件

- `/etc/nsswitch.conf` - 控制解析顺序
- `/etc/resolv.conf` - DNS服务器配置
- `/etc/hosts` - 静态主机名映射

配置DNS服务器：


```
# 方法1: 使用nmcli配置 (推荐)
nmcli connection modify "System ens3" \
    ipv4.dns "192.168.1.1 8.8.8.8"
nmcli connection up "System ens3"

# 方法2: 直接编辑配置文件 (临时, 不推荐)
vim /etc/resolv.conf
# 添加:
# nameserver 192.168.1.1
# nameserver 8.8.8.8

# 注意: 直接修改resolv.conf可能被NetworkManager覆盖!
```

⚠ 警告: 不要直接编辑/etc/resolv.conf!

```
# X 错误做法:
vim /etc/resolv.conf
# 添加: nameserver 8.8.8.8
# → NetworkManager下次更新时会覆盖你的修改!

# ✔ 正确做法:
nmcli connection modify "System ens3" ipv4.dns 8.8.8.8
nmcli connection up "System ens3"
```

配置DNS搜索域:

```
# DNS搜索域的作用:
# 当你输入"ping server1"时, 系统会自动尝试:
# 1. server1 (原始名称)
# 2. server1.example.com (添加第一个搜索域)
# 3. server1.lab.example.com (添加第二个搜索域)

# 配置搜索域:
nmcli connection modify "System ens3" \
    ipv4.dns-search "example.com lab.example.com"

# 验证配置:
nmcli connection show "System ens3" | grep dns-search
# ipv4.dns-search:                example.com
lab.example.com

# 测试:
ping server1
# → 会自动尝试 server1.example.com
```

⚠ 实际场景：配置DNS

```
# 场景：配置以下DNS设置
# 主DNS: 192.168.1.1
# 备用DNS: 8.8.8.8
# 搜索域: example.com

# 答案:
nmcli connection modify "System ens3" \
    ipv4.dns "192.168.1.1 8.8.8.8" \
    ipv4.dns-search example.com
nmcli connection up "System ens3"

# 验证:
cat /etc/resolv.conf
# nameserver 192.168.1.1
# nameserver 8.8.8.8
# search example.com

# 输出示例:
# NAME      UUID                                TYPE      DEVICE
# ens3      72ca57a2-f780-40da-b146-99f71c431e2b  ethernet  ens3
```

5.2 修改网络配置

查看连接详情：

查看特定连接详情

```
nmcli connection show ens3
```

查看IP地址

```
nmcli -p connection show ens3 | grep ipv4
```

SHELL

修改IP地址：

设置静态IP地址

```
nmcli connection mod ens3 \  
  ipv4.addresses 192.168.1.100/24 \  
  ipv4.gateway 192.168.1.1 \  
  ipv4.dns 8.8.8.8 \  
  ipv4.method manual
```

参数说明：

ipv4.addresses = 设置IP地址

ipv4.gateway = 设置网关

ipv4.dns = 设置DNS服务器

ipv4.method manual = 静态配置(非DHCP)

SHELL

重新加载连接：

重新加载配置

```
nmcli connection reload
```

重新启动连接

```
nmcli connection up ens3
```

验证新IP

```
ip addr show ens3
```

SHELL

添加新连接：

SHELL

```
# 添加新连接(使用DHCP)
nmcli connection add con-name mynet \
    type ethernet \
    ifname ens4 \
    autoconnect yes

# 参数说明：
# con-name mynet = 连接名称
# type ethernet = 连接类型
# ifname ens4 = 网络接口
# autoconnect yes = 自动连接
```

删除连接：

SHELL

```
# 删除连接
nmcli connection delete mynet

# 注意：删除活动连接会自动断开
```

相关练习：

练习名称	对应章节	说明
练习：CONFIGURING HOST NAMES AND NAME RESOLUTION	第11章 第04节	配置主机名和DNS解析
总练习：MANAGING NETWORKING	第11章	网络管理综合练习

今日总结

命令清单

服务管理：

SHELL

```
systemctl status service
systemctl start service
systemctl stop service
systemctl restart service
systemctl enable service
systemctl disable service
```

SSH:

SHELL

```
ssh user@hostname
ssh-keygen
ssh-copy-id user@hostname
```

网络配置:

SHELL

```
ip addr show
ping host
nmcli dev status
nmcli connection show
hostnamectl set-hostname name
nmtui
```

常见任务

1. 启动并设置服务开机启动

SHELL

```
systemctl start httpd
systemctl enable httpd
```

2. 配置SSH密钥认证

SHELL

```
ssh-keygen
ssh-copy-id user@server
```

3. 配置静态IP地址

SHELL

```
nmcli connection mod ens3 \  
  ipv4.addresses 192.168.1.100/24 \  
  ipv4.gateway 192.168.1.1 \  
  ipv4.method manual  
nmcli connection up ens3
```

4. 修改主机名

SHELL

```
hostnamectl set-hostname server.example.com
```

5. 禁用root SSH登录

SHELL

```
vim /etc/ssh/sshd_config  
# 修改: PermitRootLogin no  
systemctl reload sshd
```

启用允许root使用密码ssh远程访问则改为 yes